Smart Room Climate Control System

EECS 1021 Final Project Report
York University
Lassonde School of Engineering

Group Members

lailareh@my.yorku.ca
222384986
Reham
Laila

siearath@my.yorku.ca
222460208
Sieara
Thomas

1. Introduction

1.1 Problem Definition

Maintaining a comfortable indoor temperature is important in residential, educational, and workplace environments. Changes in temperature and humidity can affect indoor comfort, while manually monitoring environmental conditions may not always be efficient. An automated climate-control system can address this problem by continuously monitoring room conditions and determining when heating or cooling may be required.
This project focuses on the design and implementation of a Smart Room Climate Control System that integrates environmental sensing, embedded hardware, and Java-based software. The system uses a Seeed Studio Grove Temperature & Humidity Sensor connected to an Arduino-based system to collect real-time environmental data. These measurements are transmitted to the Java application for processing, while heating and cooling states are represented using LED indicators on the embedded system.

1.2 Project Objectives

The main objective of this project is to develop an integrated hardware-software system capable of monitoring room conditions and demonstrating automatic climate-control decisions based on temperature.
The specific objectives of the project are to:

- Measure room temperature and humidity in real time using a Seeed Studio Grove Temperature & Humidity Sensor.
- Interface the environmental sensor with an Arduino-based embedded system.
- Transmit real-time sensor readings from the Arduino to a Java application using serial communication.
- Develop a Java application capable of receiving, processing, and storing sensor data.
- Compare the measured room temperature with a desired temperature using climate-control logic.
- Determine the appropriate heating or cooling state based on temperature conditions.
- Use LED indicators to represent heating and cooling states within the prototype.
- Display environmental readings and system information through the Java console application.
- Implement error handling for incoming sensor data and communication.
- Maintain a clear separation between sensor acquisition, communication, data processing, and climate-control logic.
- Evaluate the functionality and reliability of the completed hardware-software system.

1.3 System Overview

The Smart Room Climate Control System consists of a Seeed Studio Grove Temperature & Humidity Sensor, an Arduino-based embedded system, heating and cooling LED indicators, a USB connection, and a Java application. The embedded system acquires temperature and humidity measurements from the sensor and provides the environmental data used by the system.

The Java application receives and processes sensor information and allows a desired temperature to be specified. Climate-control logic is used to compare temperature conditions and determine the appropriate system response. On the embedded side, heating and cooling states are represented through LED indicators rather than a physical fan, heater, or HVAC system.

The system therefore provides a small-scale prototype of a room climate-control system. Rather than operating high-power heating or cooling equipment, it demonstrates the sensing, data processing, communication, and decision-making processes required for a larger climate-control application.

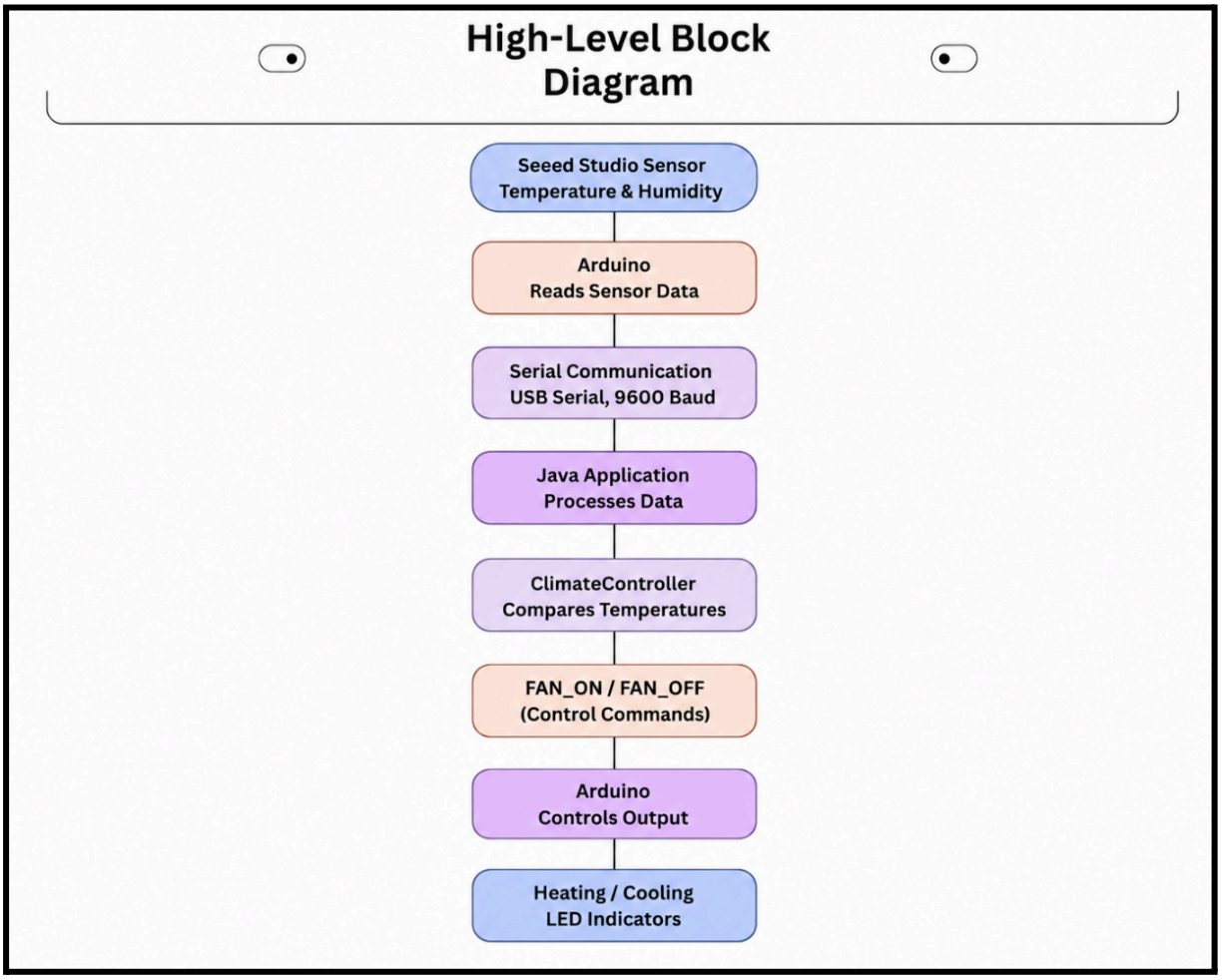
Overall, the project demonstrates an integrated hardware-software system involving real-time environmental sensing, embedded programming, serial communication, Java-based data processing, climate-control logic, and physical LED status indication. The modular separation of the embedded and Java software components also supports easier testing, maintenance, and future expansion.

2. System Architecture

The Smart Room Climate Control System is divided into two main layers: the embedded hardware layer and the Java software layer. The embedded layer is responsible for collecting temperature and humidity data from the sensor and controlling the LED indicators. The Java layer receives and processes the sensor data, compares the current temperature with the user-defined desired temperature, and determines the appropriate climate-control command. The two layers communicate through a USB serial connection, allowing sensor data to be transmitted from the Arduino to Java and control commands to be sent from Java back to the Arduino.

2.1 High-Level Block Diagram

The overall architecture of the Smart Room Climate Control System is shown in Figure 1. The system consists of a temperature and humidity sensor, an Arduino board, serial communication, the Java application, and heating and cooling LED indicators used to represent the system's climate-control states.



2.2 Hardware–Software Interaction

The Smart Room Climate Control System separates hardware-level operations from software-level processing. The Arduino interacts directly with the physical components by reading temperature and humidity measurements from the Seeed Studio Grove Temperature & Humidity Sensor and controlling the heating and cooling LED indicators.
The Java application receives sensor information through serial communication. SerialManager.java manages the serial connection, while ClimateData.java processes the incoming sensor data. The measured temperature can then be compared with the desired temperature using ClimateController.java to determine the appropriate climate-control state.
The embedded system uses LED indicators to represent heating and cooling conditions rather than controlling a physical fan, heater, or HVAC system. This separation allows the Arduino to manage direct hardware interaction while the Java application handles data processing and temperature-based control logic.

2.3 Data Flow Description

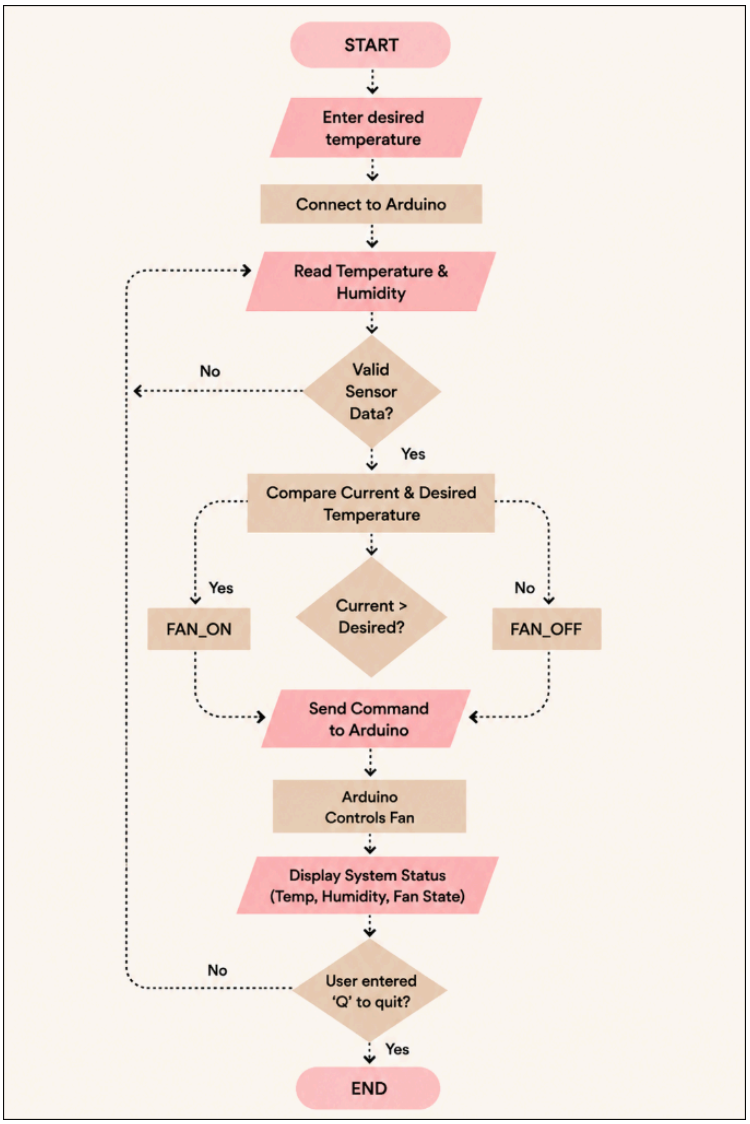
The system operates continuously by collecting environmental data and processing the measurements through the embedded and Java software layers. At the start of the Java program, the user enters a desired room temperature and the application establishes a serial connection with the Arduino.

The Seeed Studio Grove Temperature & Humidity Sensor measures the current room conditions. The Arduino reads the sensor values and transmits environmental data to the Java application through serial communication.

Within Java, SerialManager.java receives the incoming data and ClimateData.java processes the sensor information into values that can be used by the application. Error handling is included to prevent invalid sensor data from causing the Java program to terminate unexpectedly.

The measured temperature is then evaluated using the climate-control logic and compared with the desired temperature. The system determines the appropriate heating or cooling condition, while the embedded system uses LED indicators to represent the current climate-control state.

This process repeats as new sensor measurements are collected, allowing the system to continuously monitor changes in the room environment.



3. Hardware Design

The Smart Room Climate Control System uses the Seeed Studio Grove Beginner Kit for Arduino as the main hardware platform. The physical system consists of the Arduino controller, a Grove Temperature & Humidity Sensor, heating and cooling LED indicators, and a USB connection to the computer. The prototype does not use a physical cooling fan. Instead, cooling operation is represented through FAN_ON and FAN_OFF commands generated by the Java application.

3.1 Components Used and Selection

- **Arduino Board**

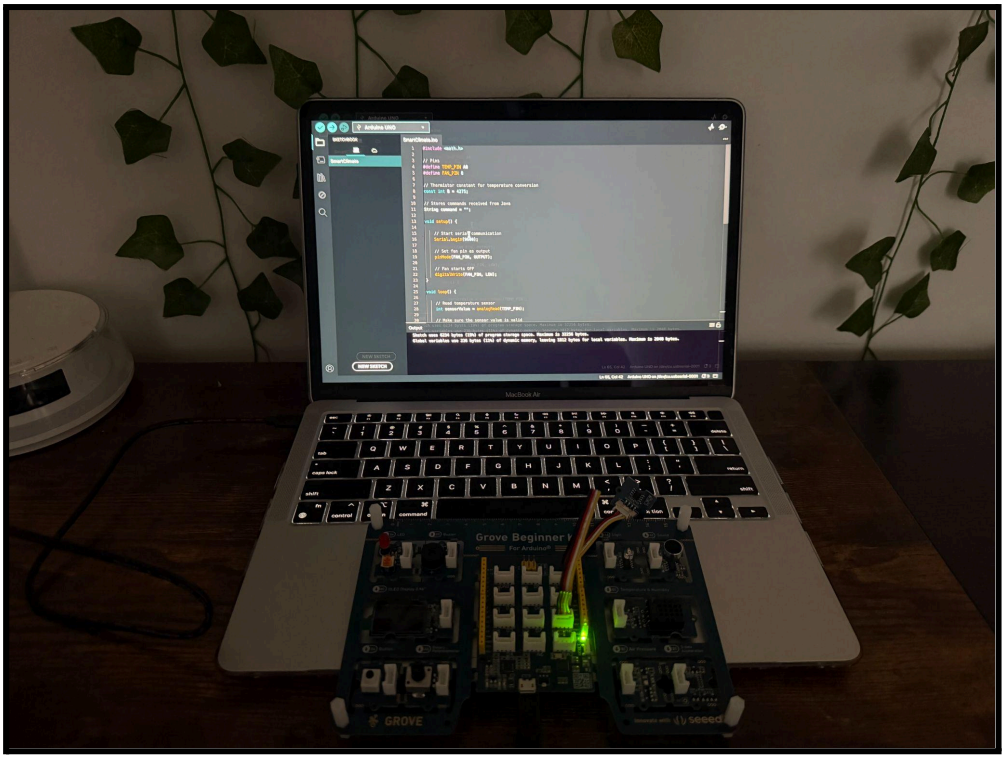
  The Arduino board serves as the main embedded controller of the system. It reads environmental data from the connected Grove sensor and sends the measurements to the Java application through serial communication. It also receives the FAN_ON and FAN_OFF commands generated by Java.
  The Arduino platform was selected because it is directly compatible with the Grove Beginner Kit and supports USB serial communication, allowing the embedded hardware to exchange information with the Java application.

- **Seeed Studio Grove Temperature & Humidity Sensor**

  The Seeed Studio Grove Temperature & Humidity Sensor is used to monitor the room environment. It provides real-time temperature and humidity measurements that are collected by the Arduino and transmitted to the Java application.
  The sensor was selected because temperature is the main environmental variable used by the climate-control algorithm. Humidity is also collected to provide additional information about the room environment.
  The Grove interface allows the sensor to integrate directly with the Grove Beginner Kit without requiring a separate breadboard or additional interface circuitry.
- **Heating & Cooling LED indicators**
  The integrated LED outputs are used to represent the heating and cooling states of the climate-control system. Rather than operating a physical heater or cooling fan, the LEDs provide a visible indication of the state selected by the embedded control logic.

- **USB Connection**

  A USB cable connects the Arduino system to the computer. The connection serves two purposes: supplying power to the embedded hardware and providing serial communication between the Arduino and Java application.

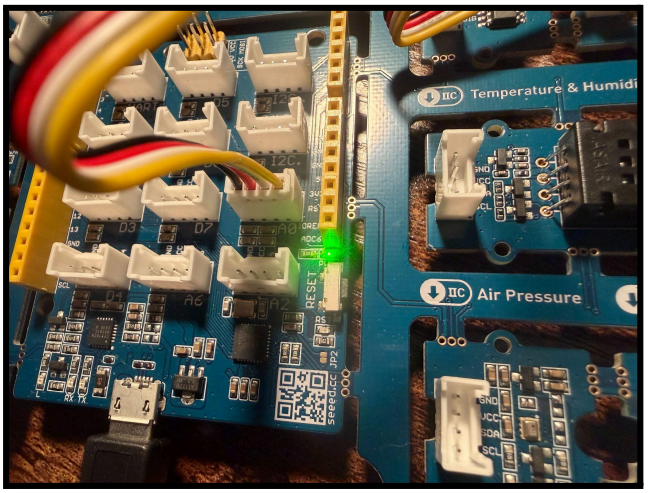


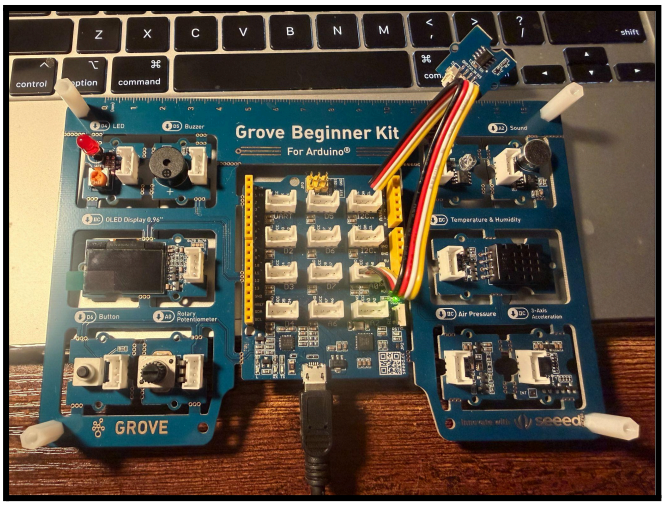
3.2 Output and Climate-Control Simulation

The prototype uses LED indicators to represent the operating state of the climate-control system. Rather than operating a physical fan, heater, or HVAC unit, the LEDs provide a visible indication of the control command received from the Java application.
When the measured temperature is greater than the user-defined desired temperature, Java generates a FAN_ON command and sends it to the Arduino. When the measured temperature is equal to or below the desired temperature, Java generates a FAN_OFF command.
Using LED indicators provides a safe, low-power method of demonstrating the climate-control response without requiring a high-power physical cooling system.

3.3 Circuit Design and Wiring

The physical hardware configuration is relatively simple because the Grove Beginner Kit provides an integrated connection platform. The Seeed Studio Grove Temperature & Humidity Sensor interfaces with the Arduino through the Grove hardware, while the Arduino is connected to the computer using USB.





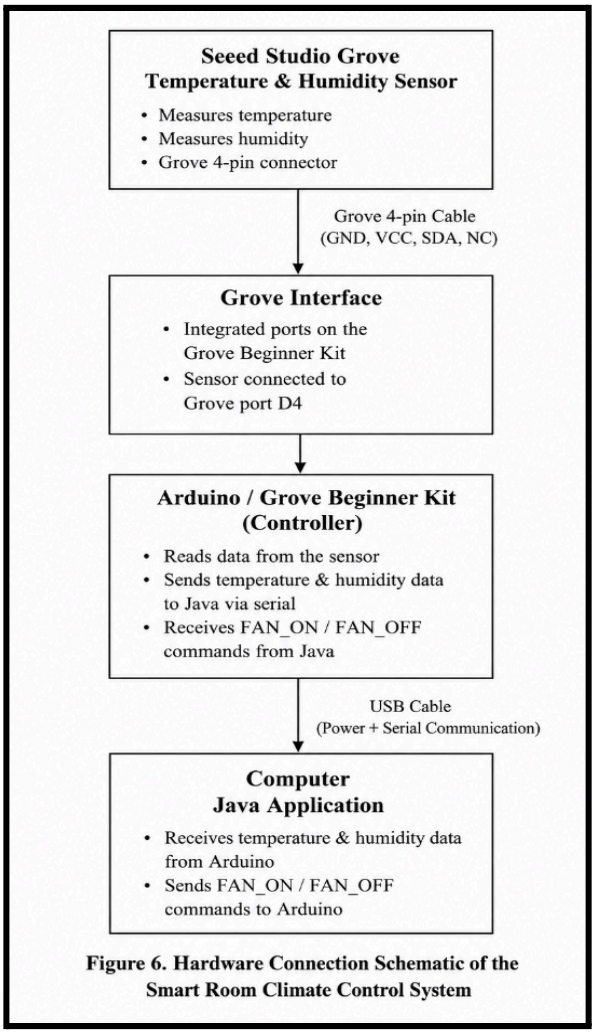


Figure 6. Hardware Connection Schematic of the Smart Room Climate Control System

3.4 Power Considerations and Constraints

The Arduino and connected Grove components are powered through the USB connection to the computer. Using USB allows a single connection to provide both electrical power and serial communication. Because the prototype only operates the Arduino, environmental sensor and low-power LED indicators, no separate actuator power supply is required. A real cooling fan, heating system, or HVAC system would have significantly different power requirements and could not be controlled directly from a microcontroller output without appropriate driver circuitry and an external power source.

3.5 Design Decisions and Trade-offs

The Grove Beginner Kit was selected because its integrated design simplifies hardware connections and eliminates the need for a separate breadboard. This allowed the project to focus on sensor acquisition, Java programming, serial communication, and system integration. A physical cooling system was not incorporated into the prototype. Heating and cooling LED indicators are used to represent the climate control state. This reduces the hardware complexity and power requirements but means the prototype cannot physically change the room temperature. The use of USB serial communication provides a simple and reliable method of transferring data between the Arduino and Java application. However, the wired connection limits portability and remote operation. A future implementation could incorporate a physical fan or HVAC interface, appropriate driver circuitry, and wireless communication to create a more complete climate-control system.

4. Software Design

The Smart Room Climate Control System uses two software layers: embedded software running on the Arduino and a Java application running on the computer. The software is divided into separate components so that sensor acquisition, serial communication, data processing, and climate-control logic can be handled independently. This modular structure makes the program easier to understand, test, and modify.

The embedded software is responsible for interacting with the temperature and humidity sensor and the physical LED indicators. The Java application handles incoming serial data and performs higher-level temperature processing and control decisions.

4.1 Embedded Software Design

The embedded software was developed using the Arduino IDE and is responsible for interacting directly with the hardware components. It collects temperature and humidity readings from the Seeed Studio Grove Temperature & Humidity Sensor and manages the heating and cooling LED indicators.

The embedded code separates sensor acquisition from climate-control logic. Environmental readings are collected and evaluated to determine the appropriate system state, while the LED indicators provide a physical representation of heating and cooling conditions. Sensor data is also transmitted through serial communication to the Java application for further processing. This modular structure keeps sensor reading, control logic, and communication organized while allowing the embedded and Java software layers to operate as separate components.

4.2 Java Application Design

The Java application handles user input, sensor-data processing, climate-control decisions, and communication with the Arduino. The program is divided into separate classes so that each major function of the system has a specific responsibility.
Main.java controls the overall program operation and allows the user to enter the desired room temperature. ClimateData.java processes incoming sensor data and stores the temperature value, while ClimateController.java compares the current temperature with the desired temperature and determines whether FAN_ON or FAN_OFF should be generated. SerialManager.java manages the serial connection between Java and the Arduino, including receiving sensor data and sending control commands.
Separating these responsibilities into individual classes improves the organization of the application and makes each component easier to test, modify, and maintain.

4.3 Libraries and Dependencies

The Java application uses the jSerialComm library to support serial communication between the computer and the Arduino. This library allows Java to identify available serial ports, establish the connection, receive sensor data, and transmit control commands.
Standard Java utilities, including Scanner, are also used to handle user input and incoming serial data. These libraries were selected because they provide the required communication and input-handling functions while keeping the application relatively simple.

4.4 Communication Handling

Communication between the Arduino and Java application occurs through USB serial communication at 9600 baud. Serial communication was selected because it provides a simple and reliable method of exchanging data between the embedded hardware and the computer while also allowing the USB connection to provide power to the system.

The Arduino transmits temperature and humidity readings to the computer as serial data. SerialManager.java receives the incoming information, while ClimateData.java parses the sensor data into values that can be processed by the application.
After the measured temperature is compared with the desired temperature, Java generates either a FAN_ON or FAN_OFF command. The selected command is transmitted back to the Arduino through SerialManager.java, allowing the embedded system to update the LED indicators.
Error handling is included to prevent invalid sensor data or connection problems from unexpectedly terminating the Java application. If incoming sensor data cannot be correctly processed, the invalid reading is ignored rather than causing the program to crash.

5. Implementation Details

The Smart Room Climate Control System was implemented by integrating the Arduino hardware with the Java application through serial communication. The implementation focuses on sensor data acquisition, temperature-based control logic, communication between the two software layers, and error handling.

5.1 Sensor and Embedded Implementation

The embedded program is responsible for interacting directly with the Seeed Studio Grove Temperature & Humidity Sensor and the LED indicators. It continuously collects temperature and humidity measurements and sends the sensor data to the Java application through the serial connection. The embedded layer does not perform the main temperature comparison used to generate the FAN_ON and FAN_OFF commands. Instead, this decision is performed by the Java application. The Arduino receives the resulting control command from Java and updates the LED indicators to represent the current climate-control state.
This separation keeps sensor acquisition and physical output control on the embedded side while higher-level temperature processing and decision-making are handled by Java.

5.2 Java Control Implementation

The Java application begins by asking the user to enter a desired room temperature. A ClimateController object stores the desired value and compares it with each valid temperature reading received from the Arduino.
When the current temperature is greater than the desired temperature, ClimateController.java generates FAN_ON. When the current temperature is equal to or below the desired temperature, it generates FAN_OFF.
The selected command is then transmitted to the Arduino through SerialManager.java, where the embedded system updates the appropriate LED output. This allows the Java application to perform the main climate-control decision while the Arduino handles sensor acquisition and physical output.

5.3 Data Parsing and Error Handling

Incoming sensor information is processed using ClimateData.fromSerial(). The method extracts the numerical temperature value from the received serial message and stores it in a ClimateData object. A try-catch structure handles incorrectly formatted data and returns null instead of allowing the application to crash.
The program also checks whether the Arduino connection is successful before beginning normal operation. If a connection cannot be established, an error message is displayed and the program terminates safely.

5.4 System Integration

During operation, Java continuously reads incoming sensor data, processes valid readings, determines the required control command, and sends the command through the serial connection. The application also displays changes in system status and provides updated system information during operation.
The user can terminate the program by entering Q. When the system shuts down, Java sends a final FAN_OFF command and disconnects from the Arduino to ensure the system stops safely.

6. System Behavior and Operation

The Smart Room Climate Control System operates continuously after the Arduino and Java application establish a serial connection. The system collects environmental data, processes the temperature reading, determines the required climate-control response, and updates the system status.

6.1 System Operation

At startup, the user enters the desired room temperature into the Java application. The program then connects to the Arduino and begins receiving sensor data. Each valid temperature reading is processed and compared with the desired temperature.
If the current temperature is greater than the desired temperature, the system generates a FAN_ON command. If the temperature is equal to or below the desired temperature, a FAN_OFF command is generated. The selected command is then sent to the Arduino through the serial connection.

6.2 Timing and Control Flow

The system operates continuously while the Java application and Arduino remain connected. The embedded program repeatedly collects new temperature and humidity measurements and transmits them through the serial connection.
The Java application continuously checks for incoming sensor readings. Each valid temperature reading is processed and compared with the desired temperature. If the required climate-control state changes, the application sends the appropriate FAN_ON or FAN_OFF command to the Arduino and displays the updated system status.
In addition to state-change updates, the Java application displays a system-status summary every 30 seconds. This summary includes the current temperature, desired temperature, fan status, and operating status.

6.3 Data Acquisition and Processing Pipeline

The complete data-processing sequence follows this order:
Sensor Reading → Arduino → Serial Communication → SerialManager.java → ClimateData.java → ClimateController.java → Control Command → Arduino
This pipeline separates sensor acquisition, communication, data processing, and control decisions into individual stages. The process continues until the user enters Q, at which point the system sends FAN_OFF, closes the serial connection, and shuts down safely.

7. Results and Evaluation

The completed prototype successfully demonstrated the main functions of the Smart Room Climate Control System. Testing was performed with the embedded hardware and Java application operating together to verify sensor-data acquisition, serial communication, temperature comparison, control-command generation, error handling, and safe shutdown.

7.1 Demonstrated Functionality

During operation, the Java application successfully received environmental data from the Arduino and processed valid temperature readings. The measured temperature was compared with the desired temperature entered by the user.
When the measured temperature was greater than the desired temperature, the system generated FAN_ON. When the temperature was equal to or below the desired temperature, the system generated FAN_OFF. The selected command was transmitted back to the Arduino, where the LED indicators represented the current system state.
The application also displayed updated system information when the control state changed and provided a system-status summary every 30 seconds.

7.2 Testing and Reliability

The system was tested by running the Arduino and Java application together and observing the response to incoming temperature readings. Testing focused on verifying sensor-data reception, temperature comparison, control-command generation, and serial communication. Basic error handling was also implemented. Invalid sensor data is detected without terminating the Java application, while an unsuccessful Arduino connection causes the program to display an error message and stop safely.

Valid sensor data	Java receives and processes the reading	Passed
Temperature above desired value	FAN_ON is generated	Passed
Temperature equal to or below desired value	FAN_OFF is generated	Passed
Invalid sensor data	Invalid reading is handled without crashing	Passed
Failed serial connection	Error is displayed and program stops safely	Passed
User enters Q	FAN_OFF is sent and connection closes safely	Passed

7.3 Evaluation

Overall, the prototype successfully demonstrated the intended hardware-software integration and automatic climate-control logic. USB serial communication provided a reliable connection for transferring sensor information and control commands between the Arduino and Java application.

The system was able to respond to changes in temperature readings and update the control state during operation. However, precise numerical measurements of communication latency, sensor accuracy, and long-term reliability were not collected during testing. Therefore, the evaluation focused primarily on functional performance and successful system operation.

The prototype is also limited by the use of LED indicators instead of a physical fan, heater, or HVAC system. As a result, the system demonstrates the decision-making and control process but does not physically regulate room temperature.

8. Strengths and Weaknesses

8.1 Strengths

A major strength of the Smart Room Climate Control System is its modular hardware-software design. Sensor acquisition, serial communication, data processing, and climate-control logic are separated into individual components, making the system easier to understand, test, and modify.
The system also provides continuous temperature monitoring and automatic control decisions based on a user-defined desired temperature. Error handling for invalid sensor data and connection failures improves the reliability of the Java application.

8.2 Weaknesses and Limitations

The main limitation is that the prototype does not control a physical fan, heater, or HVAC system. Therefore, the system demonstrates the control logic but cannot physically regulate room temperature.
The system also relies on a wired USB serial connection between the Arduino and computer, which limits portability and remote operation. Additionally, the climate-control logic is relatively simple and could be improved for more advanced real-world operation.

9. Assumptions

The system was developed under several assumptions. It is assumed that the temperature sensor is operating correctly and that the Arduino maintains a stable USB connection with the computer during operation. The measured temperature is assumed to represent the general environmental conditions around the sensor.
The prototype also assumes that FAN_ON and FAN_OFF adequately represent the control actions that would be performed by a physical cooling system. Hardware requirements associated with operating a real fan or HVAC system, such as external power supplies and driver circuitry, are therefore outside the scope of the current prototype.

10. Future Improvements

Future development could incorporate a physical fan, heater, or HVAC interface so that the system can actively regulate room temperature rather than only demonstrate control decisions. Appropriate driver circuitry and an external power source would be required to safely control higher-power equipment.
Wireless communication could also replace the USB serial connection to allow remote monitoring and control. Additional improvements could include a graphical user interface, data logging, adjustable control ranges, and more advanced climate-control algorithms.

11. Individual Contributions

The Smart Room Climate Control System was developed collaboratively by Reham and Sieara. Although each member had a primary software responsibility, both members worked together throughout the development, testing, troubleshooting, and integration of the complete system.

11.1 Reham – Arduino and Embedded System

Reham was primarily responsible for the Arduino and embedded portion of the project. Her contributions included working with the temperature and humidity sensor, developing the embedded climate-control logic, managing the heating and cooling LED indicators, and supporting the transmission of sensor information through serial communication.
She also participated in hardware setup, system testing, troubleshooting, and integration of the Arduino with the Java application.

11.2 Sieara – Java Application

Sieara was primarily responsible for the Java application. Her contributions included developing the Java classes for receiving and processing temperature sensor data, managing USB serial communication with the Arduino, comparing measured and desired temperatures, and generating the appropriate FAN_ON and FAN_OFF commands. She also contributed to user input handling, system status updates, error handling, safe shutdown and disconnection, testing, troubleshooting, and Java-Arduino integration. She also contributed to the Arduino firmware for reading and converting the analog temperature sensor data and sending readings to Java.

11.3 Collaborative Work

Both team members worked together to connect the hardware and software portions of the project, identify and resolve problems, test the completed system, and make design decisions. Although Reham focused primarily on the Arduino side and Sieara focused primarily on Java, both members contributed to and understood the overall system.